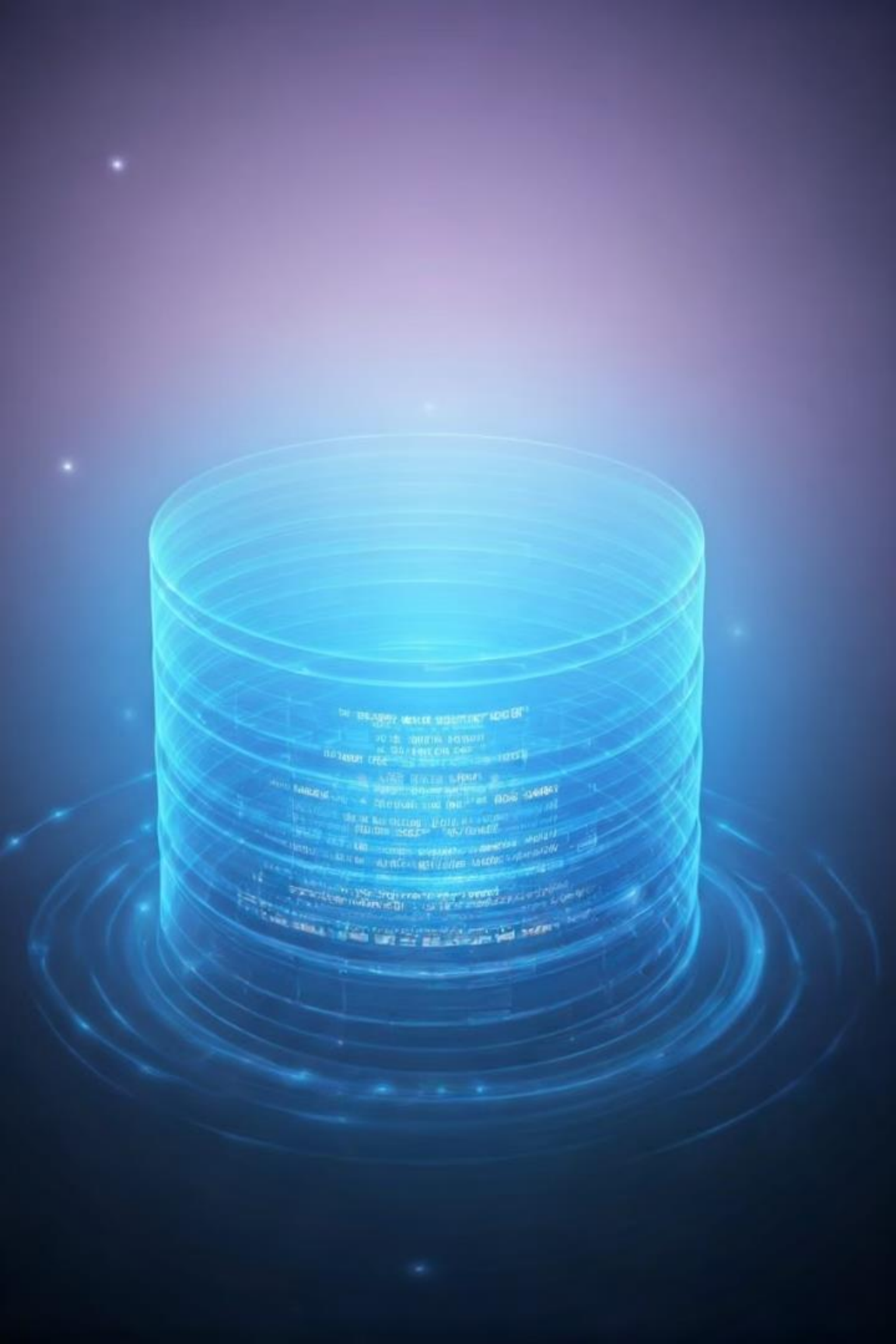




Expressões de Tabela Comuns (CTEs)

Uma visão detalhada sobre Common Table Expressions (CTEs), sua sintaxe, aplicações práticas e otimização de consultas SQL.

SM

por Salvador Melo



Definição de CTEs

O que são CTEs?

Conjuntos de resultados nomeados temporários que podem ser referenciados dentro de uma consulta SELECT, INSERT, UPDATE ou DELETE.

Em outras palavras

CTEs, ou Expressões de Tabela Comuns, são como "subconsultas nomeadas temporárias".

Entendendo CTEs

Imagine que você precisa realizar uma série de cálculos ou transformações complexas em seus dados antes de obter o resultado final da sua consulta.

Em vez de escrever tudo em uma única consulta longa e difícil de entender, você pode dividir essa lógica em partes menores e nomeadas, usando CTEs.

Cada CTE define um conjunto de resultados temporário, como se fosse uma tabela virtual, que pode ser referenciado em outras partes da sua consulta principal.

Isso torna o código mais organizado, legível e fácil de manter.

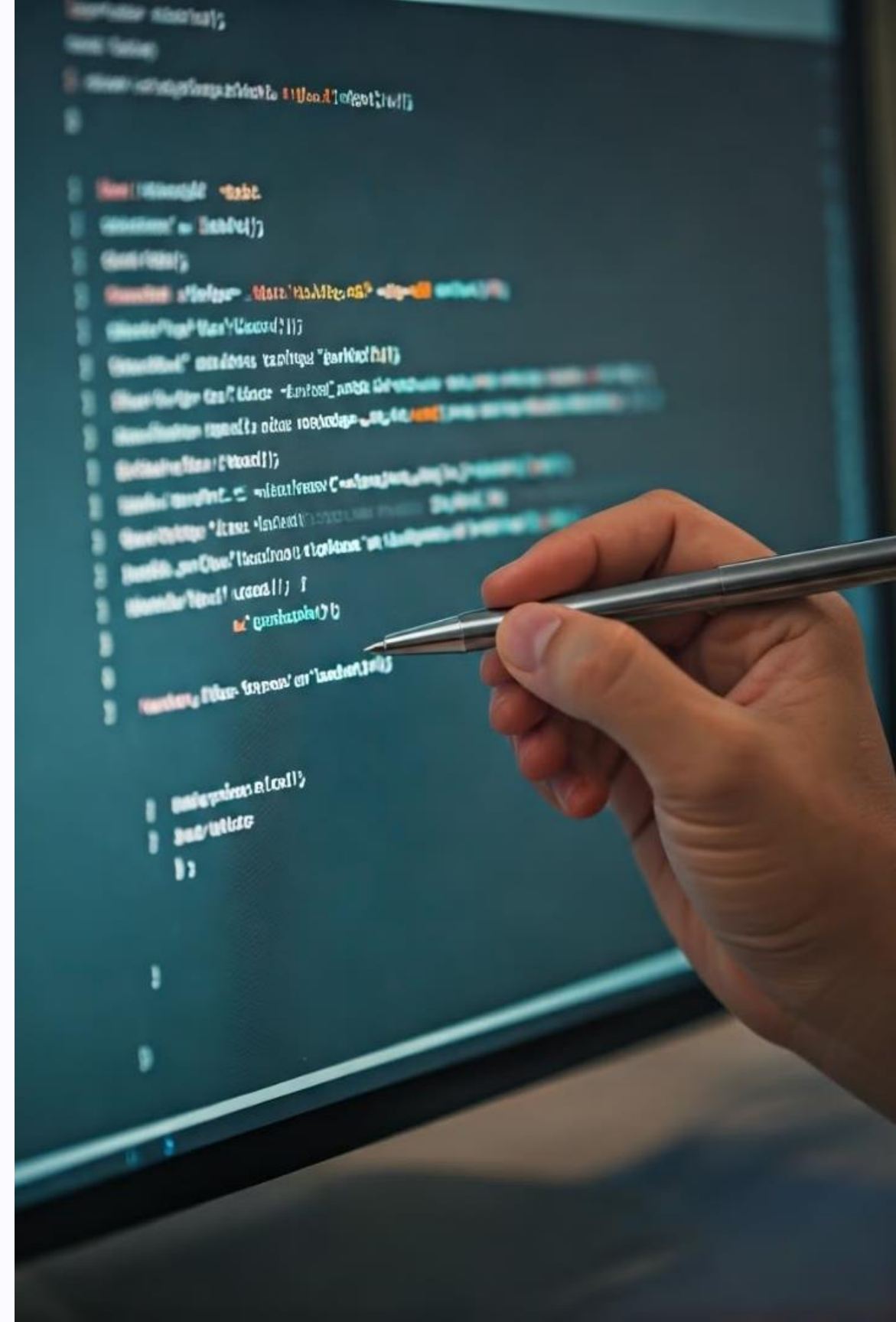
Por que você precisa saber sobre CTEs?

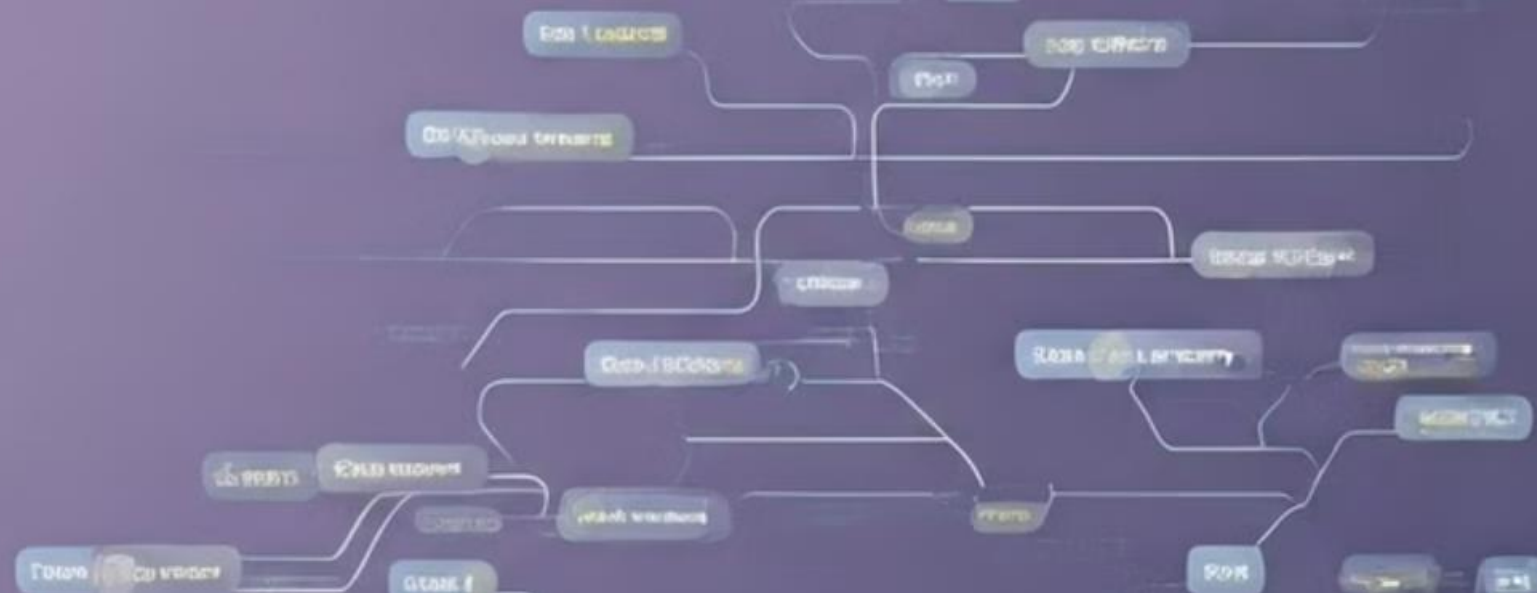
1 Melhora a Legibilidade

Consultas complexas podem se tornar extremamente confusas e difíceis de entender. CTEs permitem que você quebre a lógica em partes menores e mais fáceis de digerir, tornando o código mais legível e compreensível para você e para outros desenvolvedores.

2 Reutilização de Código

Se você precisar usar o mesmo conjunto de resultados em várias partes da sua consulta, pode definir um CTE e referenciá-lo em vários locais. Isso evita a duplicação de código e torna a manutenção mais fácil.





Mais vantagens das CTEs

1 Consultas Recursivas

CTEs são essenciais para escrever consultas recursivas, que são usadas para lidar com dados hierárquicos, como árvores de categorias, estruturas organizacionais ou redes sociais.

2 Otimização de Consultas

Em alguns casos, CTEs podem ajudar a otimizar o desempenho de consultas complexas, permitindo que o otimizador de consultas do banco de dados execute a lógica de forma mais eficiente.

Sintaxe Básica de CTEs

1

Palavra-chave **WITH**

Início da declaração CTE

2

nome_cte AS

Nome da expressão de tabela

3

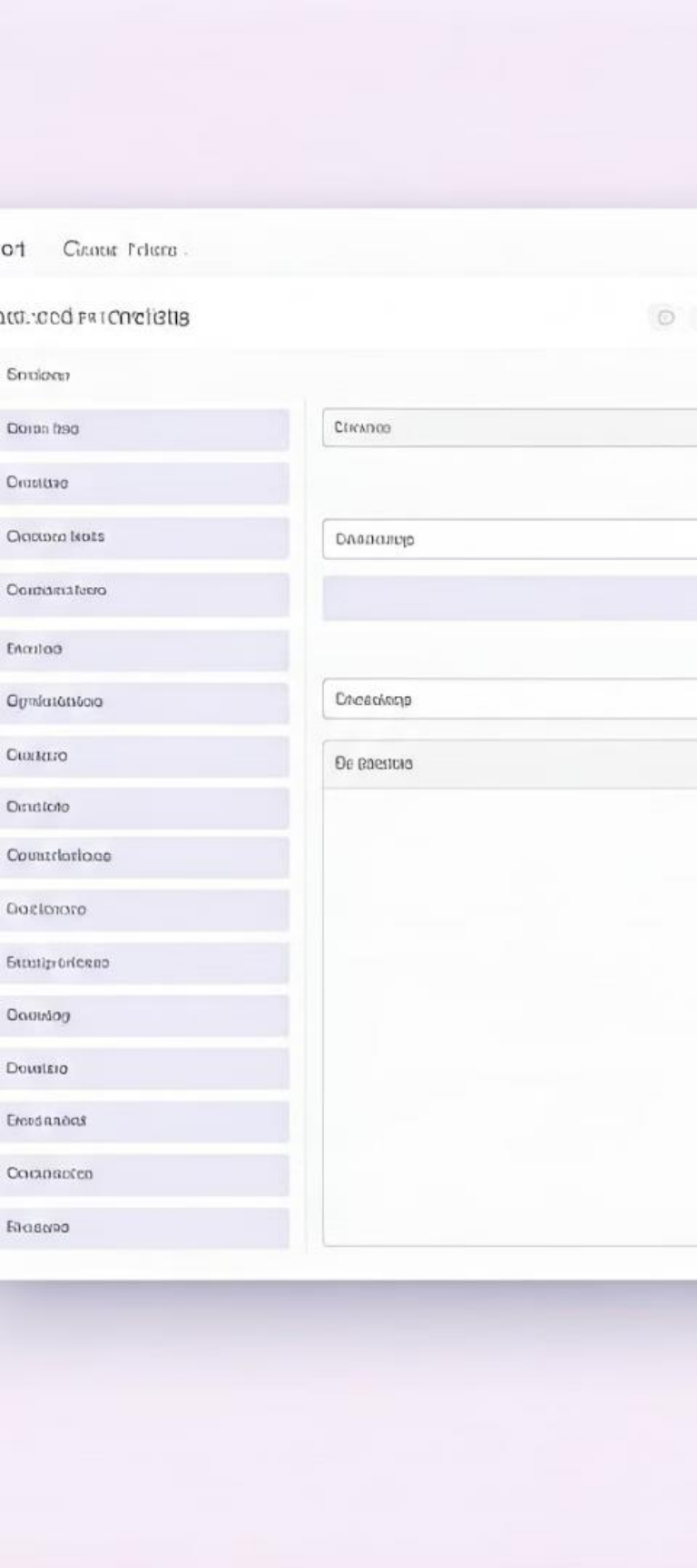
(consulta)

Consulta SQL que define o CTE

4

SELECT ... FROM nome_cte ...

Uso do CTE na consulta principal



Exemplo com Database olx_clone

```
WITH AnunciosComMensagens AS (  
    SELECT anuncio_id, COUNT(*) AS total_mensagens  
    FROM mensagens  
    GROUP BY anuncio_id  
    HAVING COUNT(*) > 1  
)  
-- Consulta final para exibir os anúncios e seus títulos  
SELECT a.titulo, acm.total_mensagens  
FROM anuncios a  
JOIN AnunciosComMensagens acm ON a.anuncio_id = acm.anuncio_id;
```


Explicação do Exemplo database: olx_clone

1

Passo 1

A CTE AnunciosComMensagens agrupa as mensagens por anuncio_id e conta o número de mensagens para cada anúncio.

2

Passo 2

Ela filtra apenas os anúncios que têm mais de uma mensagem.

3

Passo 3

A consulta final junta a CTE com a tabela anuncios para exibir o título do anúncio e o número total de mensagens.





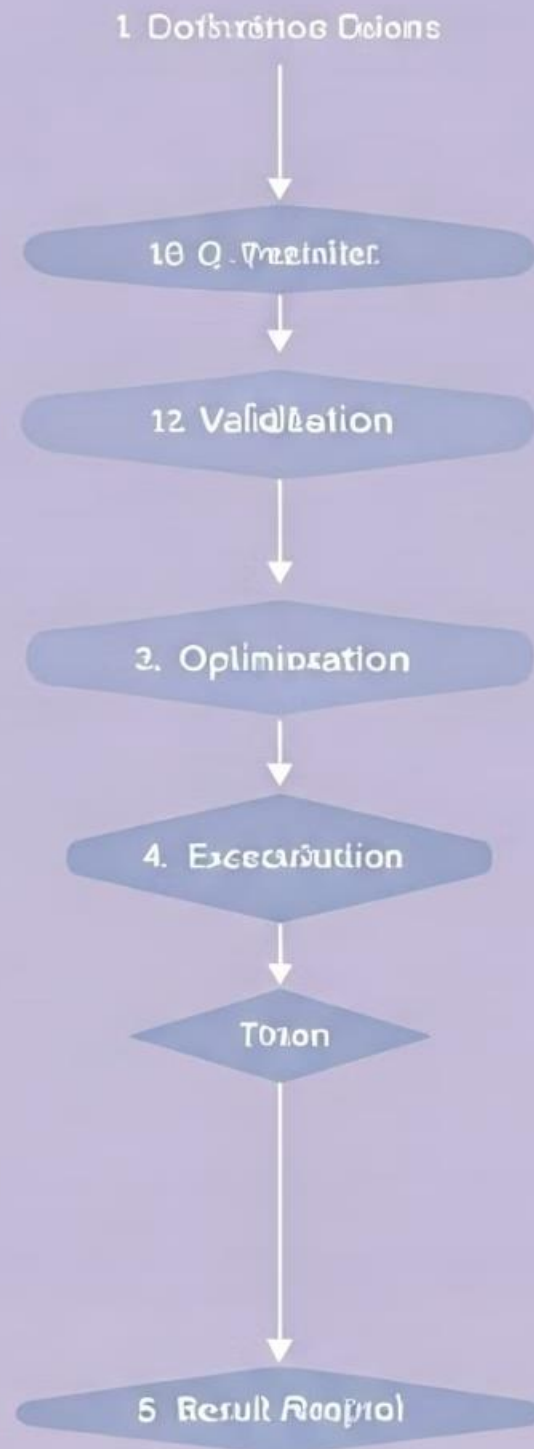
Exemplo Sem CTEs

```
-- Exemplo Sem CTEs
SELECT
    c.nome AS categoria,
    SUM(a.preco) AS total_preco,
    (SUM(a.preco) / (SELECT SUM(preco) FROM anuncios)) * 100 AS percentual
FROM anuncios a
JOIN categorias c ON a.categoria_id = c.categoria_id
GROUP BY c.nome;
```



Exemplo Com CTEs

```
-- Exemplo com CTE
/* CTE para calcular o total de preços de anúncios*/
WITH TotalPrecos AS (
    SELECT SUM(preco) AS total FROM anuncios
),
/* CTE para calcular o total de preços por categoria*/
PrecosPorCategoria AS (
    SELECT
        c.nome AS categoria,
        SUM(a.preco) AS total_preco
    FROM anuncios a
    JOIN categorias c ON a.categoria_id = c.categoria_id
    GROUP BY c.nome
)
/* Consulta final para calcular o percentual*/
SELECT
    pc.categoria,
    pc.total_preco,
    (pc.total_preco / tp.total) * 100 AS percentual
FROM PrecosPorCategoria pc, TotalPrecos tp;
```



Comentários sobre o Exemplo

1. CTE TotalPrecos:

- **Objetivo:** Calcular o preço total de todos os anúncios.
- **Resultado:** Uma tabela temporária chamada `TotalPrecos` com uma única linha e uma coluna `total` contendo o preço total de todos os anúncios.

2. CTE PrecosPorCategoria:

- **Objetivo:** Calcular o preço total dos anúncios para cada categoria.
- Os resultados são armazenados nas colunas `categoria` e `total_preco`.

Consulta Principal

- **Objetivo:** Calcular o percentual do preço total de anúncios para cada categoria em relação ao preço total de todos os anúncios.
- **Resultado:** Uma tabela final com colunas `categoria` (nome da categoria), `total_preco` (preço total dos anúncios para essa categoria) e `percentual` (percentual do preço total da categoria em relação ao preço total de todos os anúncios).



Resumo dos Benefícios de CTEs

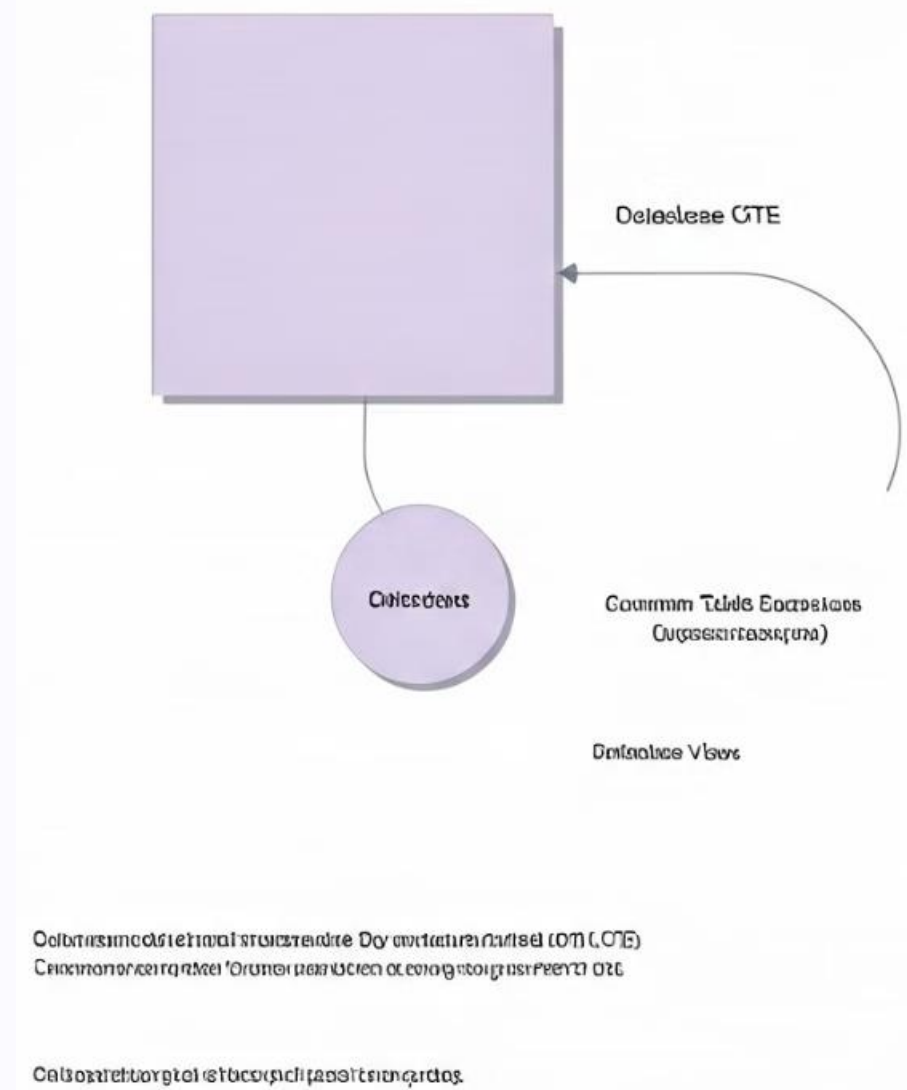
CTEs melhoram a legibilidade e organização de consultas complexas, permitindo reutilização de código e facilitando consultas recursivas.

Mesmo que aumentem o tamanho do código em consultas simples, a clareza e manutenção são priorizadas.

CTEs vs Views

Existe uma certa semelhança entre o conceito de CTEs e views?

Sim, existe uma certa semelhança entre o conceito de CTEs (Common Table Expressions) e views, mas também há diferenças importantes. Ambas as ferramentas permitem encapsular consultas complexas e reutilizar lógica SQL, mas elas servem a propósitos distintos.



Semelhanças entre CTEs e Views

Encapsulamento de Lógica

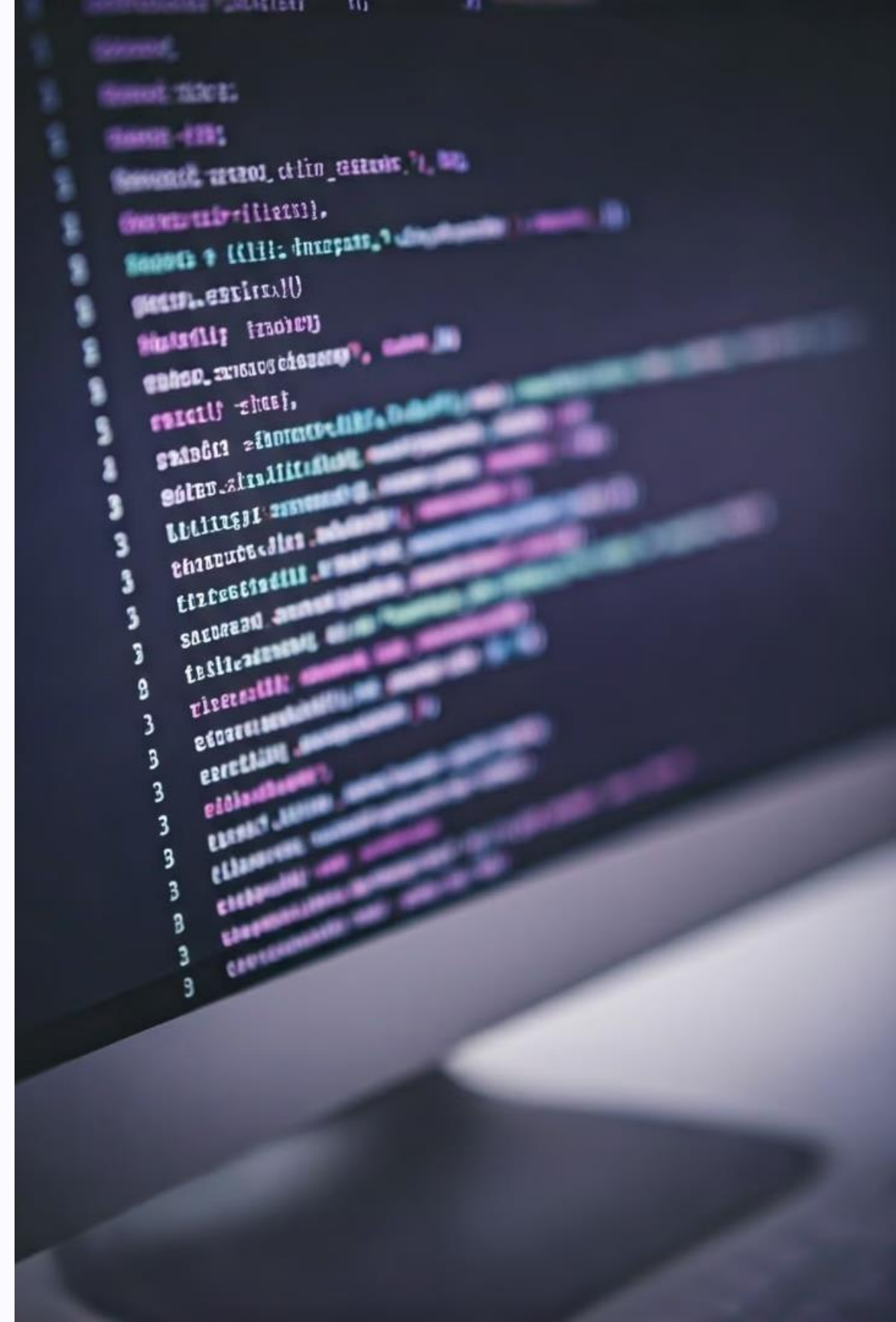
Tanto CTEs quanto views permitem que você defina uma consulta complexa e dê um nome a ela. Isso facilita a leitura e a manutenção do código SQL.

Reutilização de Código

Ambas as ferramentas permitem que você reutilize a lógica definida em vários locais da sua consulta.

Abstração de Complexidade

Ambas as ferramentas podem ser usadas para abstrair a complexidade de consultas subjacentes, tornando mais fácil para os usuários finais consultarem os dados.



Diferenças entre CTEs e Views

Temporalidade

CTEs são temporárias. Elas existem apenas durante a execução de uma única consulta. Após a conclusão da consulta, o CTE é descartado.

Views são persistentes. Elas são armazenadas no banco de dados como objetos independentes e podem ser consultadas a qualquer momento.

Escopo

CTEs têm escopo limitado à consulta em que são definidas.

Views têm escopo global dentro do banco de dados, o que significa que podem ser acessadas por qualquer usuário com permissões adequadas.

Materialização

Em muitos sistemas de banco de dados, CTEs geralmente não são materializados, o que significa que o conjunto de resultados não é armazenado em disco. Em vez disso, a consulta do CTE é executada sempre que a consulta principal é executada.

Views podem ser materializadas ou não, dependendo do sistema de banco de dados e da configuração da view. Views materializadas armazenam o conjunto de resultados em disco, o que pode melhorar o desempenho para consultas frequentes.

CTEs Recursivas: Introdução



Imagine que a tabela `categorias` do `olx_clone` tenha uma estrutura hierárquica mais complexa, onde as categorias podem ter subcategorias aninhadas em vários níveis. Por exemplo:

- Eletrônicos
- Celulares
- Smartphones
- Tablets
- Informática
- Notebooks
- Desktops

Neste cenário, podemos usar uma CTE recursiva para percorrer a hierarquia de categorias e listar todas as subcategorias de uma determinada categoria. CTEs, diferentemente das views, podem ser recursivas, permitindo a consulta de dados hierárquicos.

Em resumo: CTEs vs Views



CTEs são ideais para consultas complexas que precisam ser executadas apenas uma vez.

Views são ideais para consultas complexas que precisam ser executadas com frequência ou compartilhadas entre vários usuários.

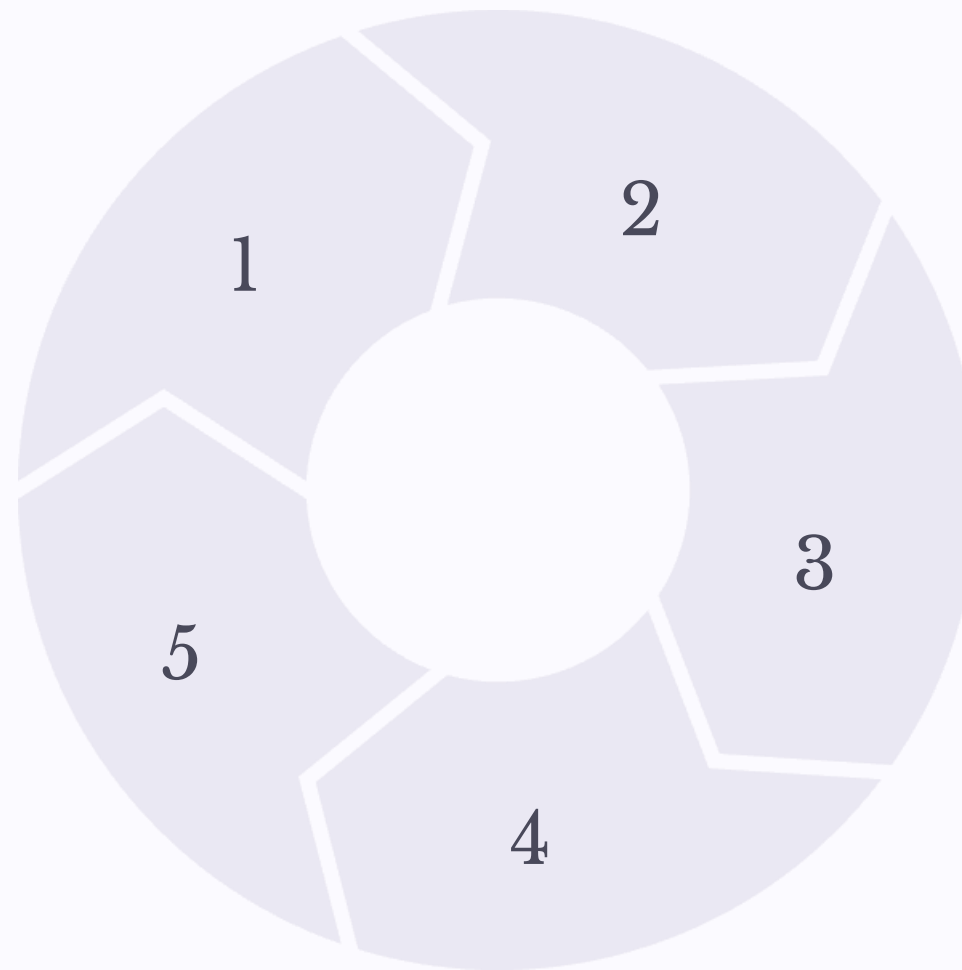
Operadores de Conjunto

Definição

Operadores que combinam os resultados de duas ou mais consultas SELECT.

EXCEPT

Retorna as linhas da primeira consulta que não estão na segunda.



UNION

Combina os resultados, removendo duplicatas.

UNION ALL

Combina os resultados, incluindo duplicatas.

INTERSECT

Retorna as linhas comuns às consultas.

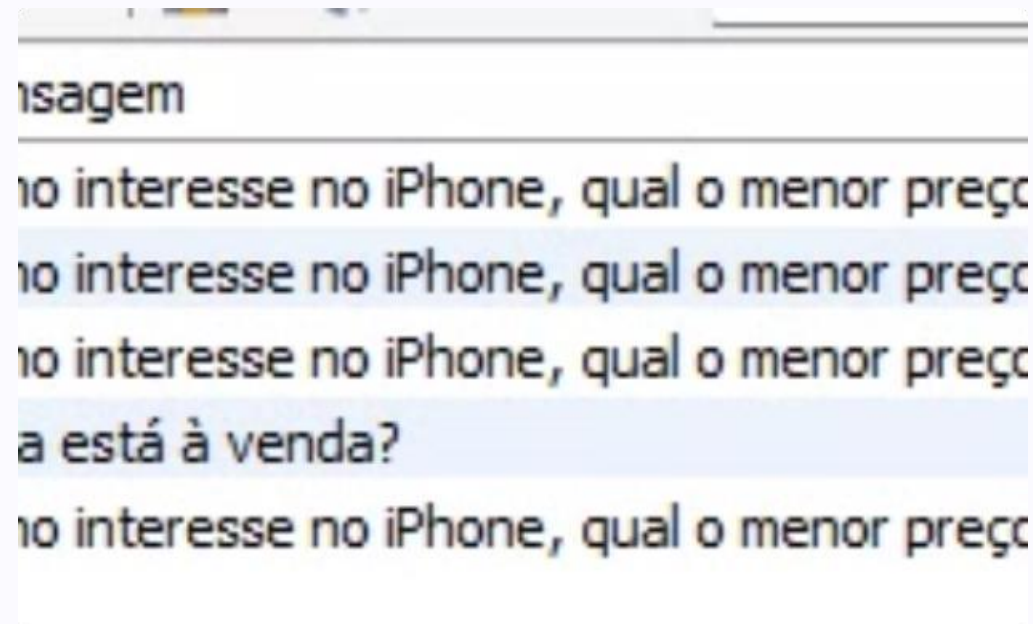
Que estruturas podem usar?

Os Operadores de Conjunto (UNION, UNION ALL, INTERSECT, EXCEPT/MINUS) não são utilizados exclusivamente com CTEs (Common Table Expressions). Eles são usados para combinar os resultados de duas ou mais consultas `SELECT` e podem ser aplicados em diversas estruturas SQL, incluindo:

- **Consultas `SELECT` Padrão:**
- **Views:**
- **Stored Procedures e Funções:**
- **Subconsultas:**
- **Triggers:**

Exemplo Prático: UNION ALL

database: olx_clone



mensagem

no interesse no iPhone, qual o menor preço

no interesse no iPhone, qual o menor preço

no interesse no iPhone, qual o menor preço

a está à venda?

no interesse no iPhone, qual o menor preço

Usando Union ALL

```
-- Usando UNION ALL
SELECT mensagem FROM mensagens WHERE remetente_id = 2
UNION ALL
SELECT mensagem FROM mensagens WHERE destinatario_id = 1;
```

Exemplo Prático: UNION

database: olx_clone

mensagem
Tenho interesse no iPhone 11
Ainda está à venda?

Usando Union ALL

```
-- Usando UNION
SELECT mensagem FROM mensagens WHERE remetente_id = 2
UNION
SELECT mensagem FROM mensagens WHERE destinatario_id = 1;
```

INTERSECT

database: exemplo_intersect.sql

Conceito Básico:

- O `INTERSECT` é usado para encontrar as linhas que são comuns a múltiplos conjuntos de resultados de consultas (query blocks). Em outras palavras, ele retorna as linhas que existem em todas as consultas especificadas.
- Ele funciona de forma similar a outras operações de conjuntos como `UNION` e `EXCEPT`.

Sintaxe:

- `query_expression_body INTERSECT [ALL | DISTINCT]`
- `query_expression_body [INTERSECT [ALL | DISTINCT]`
- `query_expression_body] [...]`

Você pode encadear múltiplos `INTERSECT`s para encontrar a interseção de mais de dois conjuntos de resultados.

Observações

- Se você não especificar `ALL` ou `DISTINCT`, o padrão é `DISTINCT`.
- `DISTINCT`: Remove linhas duplicadas do resultado final, garantindo que cada linha seja única.
- `ALL`: Retorna todas as linhas correspondentes, incluindo duplicatas.
- As consultas que compõem a operação `INTERSECT` devem ter o mesmo número de colunas.
- Os tipos de dados das colunas correspondentes nas diferentes consultas devem ser compatíveis.

Exemplo Intersect

```
create database if not exists exemplo_intersect;
-- Tabela de alunos aprovados
CREATE TABLE alunos_aprovados (
  id INT PRIMARY KEY,
  nome VARCHAR(255)
);

-- Tabela de alunos participantes
CREATE TABLE alunos_participantes (
  id INT PRIMARY KEY,
  nome VARCHAR(255)
);
```

```
-- Inserindo alguns dados de exemplo
INSERT INTO alunos_aprovados (id, nome) VALUES
(1, 'Alice'),
(2, 'Bob'),
(3, 'Carol'),
(4, 'David');

INSERT INTO alunos_participantes (id, nome) VALUES
(2, 'Bob'),
(3, 'Carol'),
(5, 'Eve');
```

SELECT nome FROM alunos_aprovados

INTERSECT

SELECT nome FROM alunos_participantes;

Comparando alternativas

Usando <code>INTERSECT</code> :	Usando <code>WHERE</code> e subconsulta:	Usando <code>JOIN</code> :
<pre>SELECT nome FROM alunos_aprovados INTERSECT SELECT nome FROM alunos_participantes;</pre>	<pre>SELECT nome FROM alunos_aprovados WHERE nome IN (SELECT nome FROM alunos_participantes);</pre>	<pre>SELECT DISTINCT a.nome FROM alunos_aprovados a JOIN alunos_participantes p ON a.nome = p.nome;</pre>

As três consultas retornam o mesmo resultado, mas cada uma tem suas próprias vantagens e desvantagens em termos de legibilidade, desempenho e flexibilidade.

Em resumo:

- Se você precisa expressar claramente a interseção de conjuntos de resultados e a legibilidade é uma prioridade, `INTERSECT` pode ser uma boa opção.
- Se você precisa de mais flexibilidade para expressar condições de filtragem complexas ou se estiver usando uma versão de banco de dados que não suporta `INTERSECT`, `WHERE` ou `JOIN` são alternativas viáveis.



MySQL e Operadores de Conjunto

8.0.31

Versão MySQL

A partir do MySQL 8.0.31, o MySQL introduziu suporte para os operadores INTERSECT e EXCEPT padrão SQL.

Isso significa que, nessas versões mais recentes, você pode usar diretamente a palavra-chave INTERSECT para encontrar as linhas comuns entre duas ou mais instruções SELECT.

2

Operadores Adicionados

INTERSECT e EXCEPT são implementados conforme o padrão SQL nas versões mais recentes.



Regras para Uso de Operadores de Conjunto

1 Compatibilidade de Consultas

As consultas devem ter o mesmo número de colunas e tipos de dados compatíveis.

2 Ordem das Colunas

A ordem das colunas nas consultas deve corresponder para combinação correta dos resultados.

3 Nomes de Colunas

Os nomes das colunas no resultado final são determinados pelos nomes na primeira consulta.

Otimização de Consultas





Reescrita de Consultas para Otimização



Índices

Utilização de índices adequados.



Cláusulas WHERE

Otimização de cláusulas WHERE (ex: evitar funções em colunas indexadas).



JOINs

Reescrita de subconsultas em JOINs.



CTEs

Utilização de CTEs para melhorar a legibilidade e desempenho.

Mais Técnicas de Otimização

Reduzir "SELECT *"

Selecione apenas as colunas necessárias para reduzir o tráfego de dados e melhorar o desempenho.

Otimização de JOIN

Escolha o tipo correto de JOIN e certifique-se de que as colunas de junção estão indexadas.

Estatísticas Atualizadas

Manter estatísticas atualizadas nas tabelas.

Hardware Adequado

Utilizar hardware adequado para o banco de dados.

